

WiSARD, DRASiW e Bleaching

Redes Neurais Sem-Peso — Fundamentos e Extensões

Prof. Diego Carvalho 

diego.carvalho@cefet-rj.br

Cefet/RJ

31 de maio de 2026

Roteiro da Aula (3h)

🕒 Parte I — 50min

Motivação e História
Neurônio RAM
Discriminador WiSARD
Multi-discriminador
Atividade 1

🕒 Parte II — 60min

Problema de saturação
DRASiW: contadores
Imagens mentais
Regras fuzzy
Atividade 2

🕒 Parte III — 60min

Sobretreinamento
Bleaching sequencial
B-bleaching binário
Comparativo
Atividade 3 + Discussão

Intervalo de 10min entre cada parte

Parte I

Contextualização e Motivação

O que são Redes Neurais Sem-Peso?

Paradigma alternativo ao modelo de McCulloch-Pitts

- Neurônios clássicos operam com **pesos sinápticos**
- Neurônios sem-peso operam com **memórias RAM endereçáveis**
- Aprendizado = **escrita em memória** (sem gradiente, sem backprop)

“The dendritic tree topology — a quite noticeable morphological feature of biological neurons — is central to the model.” — França et al. (ICARCV 2010)

Neurônio Clássico

entrada × **peso** → soma → ativação

Neurônio RAM

endereço → **lookup** → saída {0,1}

Treinamento: escrita de 1s

Classificação: leitura + soma

Breve História

Ano	Marco
~1959	Bledsoe & Browning — primeiros n-tuplos RAM em reconhecimento de padrões
1966	Aleksander — SLAM (<i>Stored Logic Adaptive Microcircuit</i>)
1984	Wilkie, Stonham & Aleksander — WiSARD patenteado e comercializado
1990	Aleksander & Morton — <i>An Introduction to Neural Computing</i>
1998	Soares et al. — DRASiW: retroclassificação e imagens mentais
2010	Grieco et al. — DRASiW + imagens fuzzy (MNIST)
2013	Carvalho et al. — B-bleaching: avoidance ágil de sobretreinamento

Por que estudar WiSARD em 2026?

Por que estudar WiSARD em 2026?

- **Velocidade extrema:** treinamento e classificação em tempo real sem multiplicações

Por que estudar WiSARD em 2026?

- **Velocidade extrema:** treinamento e classificação em tempo real sem multiplicações
- **Paralelismo natural:** cada discriminador opera de forma independente

Por que estudar WiSARD em 2026?

- **Velocidade extrema:** treinamento e classificação em tempo real sem multiplicações
- **Paralelismo natural:** cada discriminador opera de forma independente
- **Aprendizado online:** novos padrões são incorporados incrementalmente

Por que estudar WiSARD em 2026?

- **Velocidade extrema:** treinamento e classificação em tempo real sem multiplicações
- **Paralelismo natural:** cada discriminador opera de forma independente
- **Aprendizado online:** novos padrões são incorporados incrementalmente
- **Interpretabilidade:** as memórias RAM têm semântica direta (contagens de subpadrões)

Por que estudar WiSARD em 2026?

- **Velocidade extrema:** treinamento e classificação em tempo real sem multiplicações
- **Paralelismo natural:** cada discriminador opera de forma independente
- **Aprendizado online:** novos padrões são incorporados incrementalmente
- **Interpretabilidade:** as memórias RAM têm semântica direta (contagens de subpadrões)
- **Aplicações ativas:** visão computacional, bioinformática, PLN, controle de robôs



Parte I — WiSARD

O Neurônio RAM e o Discriminador

O Neurônio RAM

Estrutura:

- Uma RAM com n **bits** de endereço
- Capacidade de 2^n posições (cada uma: 0 ou 1)
- Recebe como entrada uma **n-tupla** — subconjunto de n bits do padrão de entrada

Treinamento:

1. Inicializar todas as posições com **0**
2. Para cada padrão de treino: escrever **1** na posição endereçada pela n-tupla

Classificação:

- Ler o conteúdo da posição endereçada pela n-tupla de teste
- Saída: **0** ou **1**

RAM com $n=3$ bits de endereço

Endereço	Conteúdo
000	0
001	0
010	1 ← treinado
011	0
100	1 ← treinado
101	0
110	0
111	1 ← treinado

O Mapeamento Pseudo-aleatório

Problema: como distribuir os bits do padrão de entrada pelas n -tuplas?

A entrada binária de $X \times n$ bits é **embaralhada** por uma função bijetora ε (shuffling):

$$\varepsilon : \{0, 1\}^{X \cdot n} \rightarrow \{0, 1\}^{X \cdot n}$$

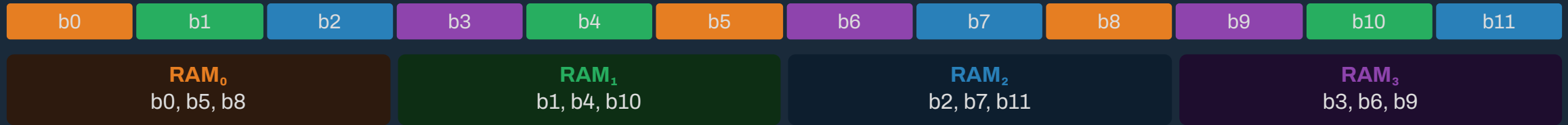
- Cada RAM i recebe exatamente n bits distintos
- A partição é **fixa** após a inicialização
- Normalmente gerada de forma **pseudo-aleatória**

Observações importantes:

- Pixels espaçados capturam correlações globais
- O mapeamento influencia diretamente a capacidade de generalização
- Mapeamentos não-aleatórios (guiados pelo domínio) podem melhorar o desempenho

O Mapeamento Pseudo-aleatório

Exemplo: entrada de 12 bits → 4 RAMs com $n=3$



O Discriminador WiSARD

Um **discriminador** é um conjunto de X neurônios RAM que cobre **todo o campo de entrada**:

$$\text{Resposta } r = \sum_{i=1}^X \text{RAM}_i(\text{n-tupla}_i)$$

Propriedades de r :

- $r = X \rightarrow$ padrão idêntico ao treinamento
- $r = 0 \rightarrow$ nenhuma n-tupla foi vista antes
- $0 < r < X \rightarrow$ similaridade parcial (**generalização**)

Tradeoff fundamental:

- n grande $\rightarrow$ mais especialização, menos generalização
- n pequeno $\rightarrow$ mais generalização, menos discriminação

Fase de classificação (exemplo)

RAM₀: endereço 010 $\rightarrow$ 1

RAM₁: endereço 101 $\rightarrow$ 0

RAM₂: endereço 110 $\rightarrow$ 1

RAM₃: endereço 011 $\rightarrow$ 1

$$r = 3 / 4 = 75\%$$

WiSARD Multi-discriminador

Para classificação em m **classes**, usa-se um discriminador por classe:



Confiança da classificação:

$$c = \frac{d}{r_T} = \frac{r_T - r_I}{r_T}$$

onde r_T é a maior resposta e r_I é a segunda maior.

Algoritmo de Treinamento (WiSARD)

```
1 def treinar(wisard, padrao_binario, classe):
2     """
3     padrao_binario : lista de 0s e 1s, tamanho = X * n
4     classe         : rótulo da classe (índice do discriminador)
5     """
6     discriminador = wisard.discriminadores[classe]
7
8     for i, ram in enumerate(discriminador.rams):
9         # Extrai a n-tupla correspondente (após shuffling)
10        n_tupla = extrair_n_tupla(padrao_binario, wisard.mapeamento, i)
11        # Converte n-tupla para endereço inteiro
12        endereco = tupla_para_int(n_tupla)
13        # Escreve 1 na posição (aprendizado)
14        ram[endereco] = 1
```

Algoritmo de Classificação (WiSARD)

```
1 def classificar(wisard, padrao_binario):
2     respostas = {}
3     for classe, discriminador in wisard.discriminadores.items():
4         r = 0
5         for i, ram in enumerate(discriminador.rams):
6             n_tupla = extrair_n_tupla(padrao_binario, wisard.mapeamento, i)
7             r += ram[tupla_para_int(n_tupla)] # soma as saídas
8         respostas[classe] = r
9     return max(respostas, key=respostas.get) # classe com maior r
```

Exemplo Numérico — Treinamento

Configuração: $X = 4$ RAMs, $n = 3$ bits, imagem 4×3 pixels

Padrão de treino da Classe A (linha vertical):

```
1 0 0
1 0 0
1 0 0
1 0 0
```

Representação linear: 1 0 0 1 0 0 1 0 0 1 0 0

Após shuffling (exemplo):

- $RAM_0 \leftarrow \text{bits } \{0, 4, 8\} \rightarrow 1\ 0\ 0 \rightarrow \text{endereço } 100\ (4)$
- $RAM_1 \leftarrow \text{bits } \{1, 5, 9\} \rightarrow 0\ 0\ 0 \rightarrow \text{endereço } 000\ (0)$
- $RAM_2 \leftarrow \text{bits } \{2, 6, 10\} \rightarrow 0\ 0\ 0 \rightarrow \text{endereço } 000\ (0)$
- $RAM_3 \leftarrow \text{bits } \{3, 7, 11\} \rightarrow 1\ 0\ 1 \rightarrow \text{endereço } 101\ (5)$

Escrevemos **1** nas posições 4, 0, 0, 5 das respectivas RAMs.

Estado das RAMs após treino

Endereço	RAM_0	RAM_1	RAM_2	RAM_3
000 (0)	0	1	1	0
001 (1)	0	0	0	0
010 (2)	0	0	0	0
011 (3)	0	0	0	0
100 (4)	1	0	0	0
101 (5)	0	0	0	1
110 (6)	0	0	0	0
111 (7)	0	0	0	0

r (padrão exato) = $4/4 = 100\%$

Atividade 1 — WiSARD na Mão

Duração: 15 minutos | Em duplas

Configuração: WiSARD com $X = 4$ RAMs, $n = 2$ bits, entrada de 8 bits

Mapeamento (fixo):

RAM	Bits da entrada
RAM ₀	b0, b3
RAM ₁	b1, b6
RAM ₂	b4, b7
RAM ₃	b2, b5

Padrões de treino:

- Classe A: 1 0 0 1 1 0 0 1
- Classe B: 0 1 1 0 0 1 1 0

Tarefas:

1. Preencha as tabelas das 4 RAMs para cada discriminador (A e B)
2. Classifique o padrão de teste: 1 0 1 1 1 0 1 0
3. Calcule a resposta r e a confiança c para cada discriminador

Parte II — DRASiW

Retroclassificação e Imagens Mentais

O Problema da Saturação

Com muitos padrões de treinamento, as RAMs ficam com **quase todas as posições em 1**:

O que acontece:

- À medida que mais padrões são apresentados, mais posições das RAMs são marcadas com 1
- Com muitos padrões, a maioria das posições $\rightarrow 1$
- **Consequência:** $r \approx X$ para qualquer entrada
- Os discriminadores produzem **respostas empatadas** (ties)
- A confiança $c \rightarrow 0$, tornando a classificação inútil

Quando ocorre:

- Dataset de treino grande
- n pequeno (RAM com poucas posições)

- Classes com alta variabilidade interna

RAM saturada (antes)



r para qualquer entrada $\approx X$
discriminadores empatados $\rightarrow c \approx 0$

Informação perdida:

A RAM não distingue subpadrões frequentes dos raros

DRASiW — Ideia Fundamental

DRASiW (*WiSARD* ao contrário) substitui os bits 0/1 por **contadores inteiros**:

Modificação no treinamento:

Em vez de escrever **1**, **incrementa** o contador da posição:

$$\text{RAM}_i[\text{endereço}] += 1$$

Resultado após treinamento:

Cada posição guarda a **frequência** com que aquele subpadrão foi apresentado.

- Valor alto → subpadrão **representativo** da classe
- Valor baixo → subpadrão **raro** ou accidental

Classificação: funciona igual ao *WiSARD* original
(conta posições $\neq 0$)

WiSARD vs DRASiW

Endereço	WiSARD	DRASiW
000	1	7
001	0	0
010	1	5
011	1	2
100	1	7
101	0	0
110	1	6
111	1	1

7 padrões de treino

Máx = 7 (ocorreu em todos)

Imagens Mentais — O Processo DRASiW

A operação **inversa** (retroclassificação) produz um protótipo da classe:

Procedimento para construir a “imagem mental”:

Para cada RAM j e cada bit b_{ij} da n -tupla:

$$B_{ij} = \sum_{\text{endereços onde } b_i=1} \text{RAM}_j[\text{endereço}]$$

Intensidade do pixel associado ao bit b_{ij} proporcional a B_{ij} :

$$\text{pixel}_{ij} \propto B_{ij}$$

Intuição: pixels que aparecem frequentemente nos padrões de treino terão valores B maiores → mais escuros na imagem mental.

Linha vertical: "imagem mental"



Coluna esquerda = pixel frequente
Colunas c/d = pixels raros

→ Protótipo da Classe "Linha Vertical"

Grieco et al. (Neurocomputing, 2010) demonstraram isso no banco MNIST com dígitos manuscritos.

DRASiW - Exemplo

Grieco et al. (Neurocomputing, 2010) demonstraram isso no banco MNIST com dígitos manuscritos.



Fig. 4. “Mental” images of the MNIST handwritten numbers training set.

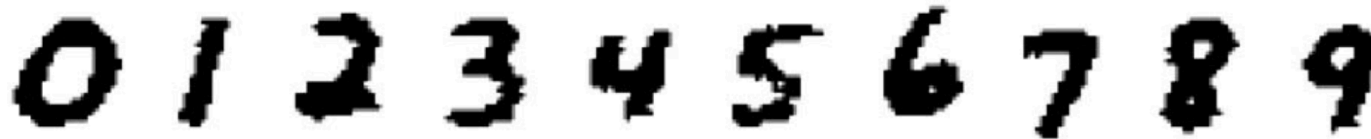
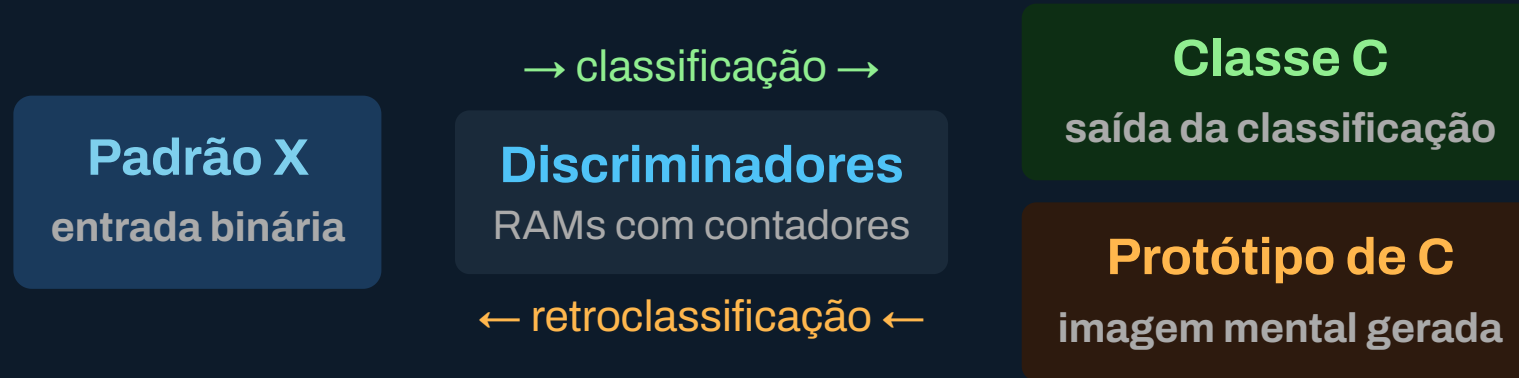


Fig. 5. Prototypes generated via DRASiW’s “mental” images.



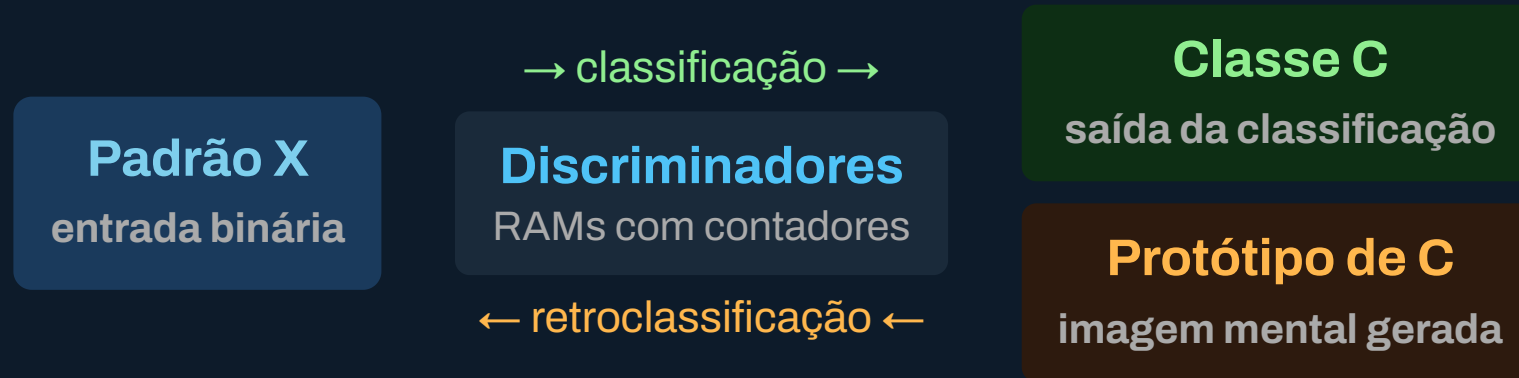
Fig. 6. Prototypes generated via “mental” images of fuzzy discriminators.

DRASiW — Bidireccionalidade



Propriedades do DRASiW:

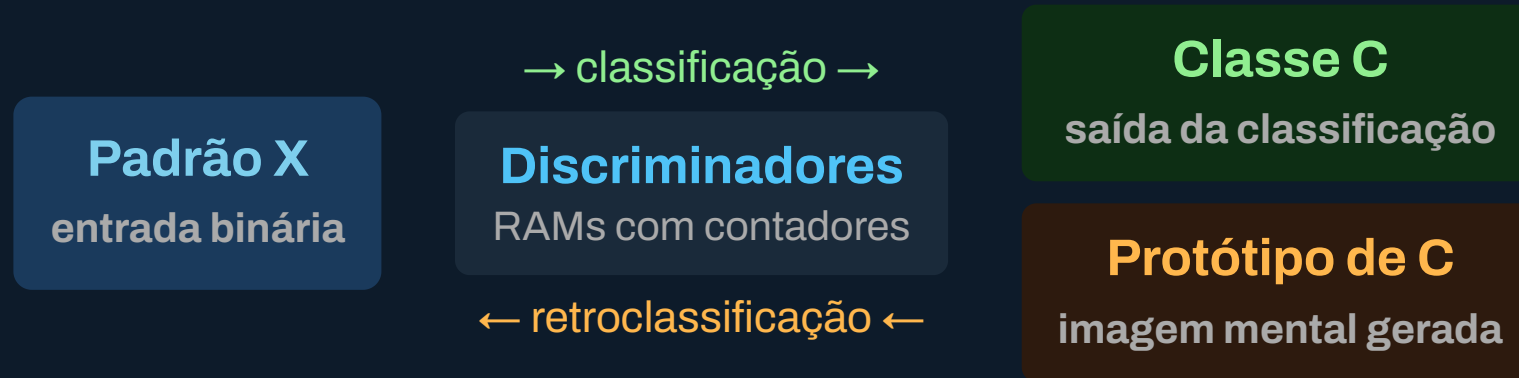
DRASiW — Bidireccionalidade



Propriedades do DRASiW:

- O protótipo **não precisa ser idêntico** a nenhum padrão do conjunto de treino

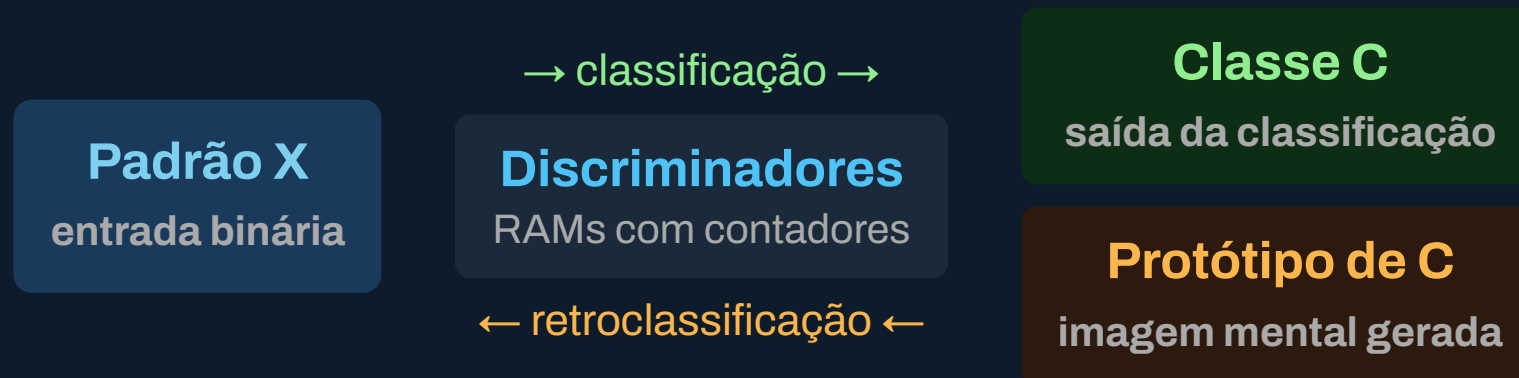
DRASiW — Bidireccionalidade



Propriedades do DRASiW:

- O protótipo **não precisa ser idêntico** a nenhum padrão do conjunto de treino
- É considerado válido se for **corretamente classificado** pelo multi-discriminador

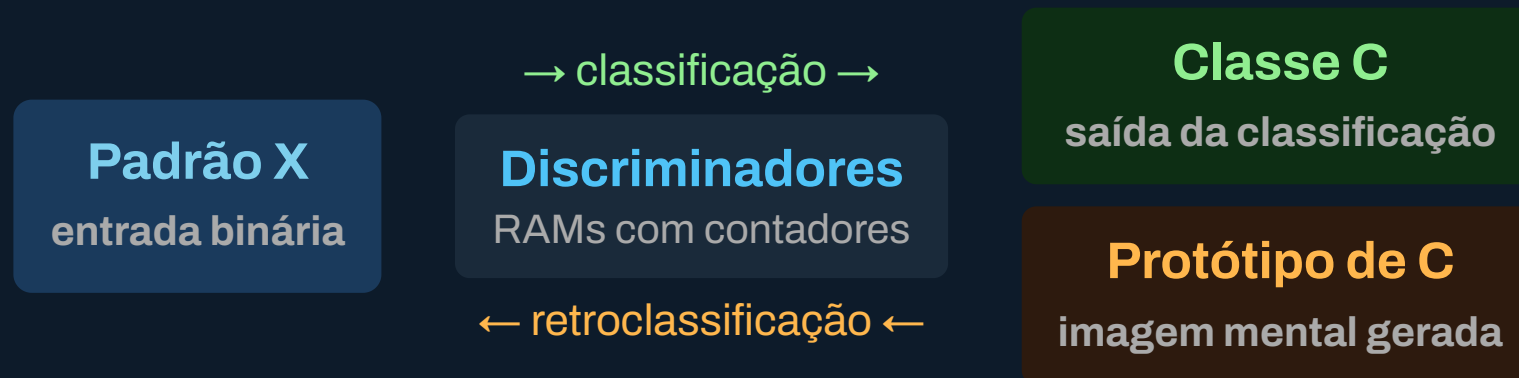
DRASiW — Bidireccionalidade



Propriedades do DRASiW:

- O protótipo **não precisa ser idêntico** a nenhum padrão do conjunto de treino
- É considerado válido se for **corretamente classificado** pelo multi-discriminador
- Permite **inspeção e interpretação** do que o modelo aprendeu

DRASiW — Bidireccionalidade



Propriedades do DRASiW:

- O protótipo **não precisa ser idêntico** a nenhum padrão do conjunto de treino
- É considerado válido se for **corretamente classificado** pelo multi-discriminador
- Permite **inspeção e interpretação** do que o modelo aprendeu
- Análogo (não idêntico) às **Bidirectional Associative Memories** (BAM) de Kosko

DRASiW como Regras Fuzzy

Uma formulação equivalente interpreta os contadores como **graus de pertinência fuzzy**:

Cada RAM j define uma **regra fuzzy OR** (t-conorma):

$$\text{saída}_j = \bigvee_{\text{endereços ativos}} \mu_j(\text{endereço})$$

onde $\mu_j(\text{endereço}) = \frac{\text{contador}}{\max(\text{contadores})}$

Vantagem: saída semântica compatível com sistemas de raciocínio simbólico

Propriedades:

- Equivalente ao DRASiW em termos de protótipos gerados
- Fornece **regras legíveis** em vez de imagens
- Permite **pipeline** com outros sistemas baseados em

Exemplo — RAM₀ com n=2

a	b	Σ	μ (norm.)
0	0	0	0.0
0	1	2	0.4
1	0	1	0.2
1	1	5	1.0

Regra: SE (a=1 E b=1) ENTÃO ativação máxima

Atividade 2 — Construindo uma Imagem Mental

Duração: 15 minutos | Individual

Dados: Discriminador treinado com 5 padrões da Classe “X” (4 pixels, n=2, X=2 RAMs)

Mapeamento: $RAM_0 \leftarrow \{b0, b1\}$, $RAM_1 \leftarrow \{b2, b3\}$

Conjunto de treino (Classe “X”):

Padrão	b0	b1	b2	b3
P1	1	0	1	0
P2	1	1	1	1
P3	1	0	0	1
P4	0	1	1	0
P5	1	1	1	0

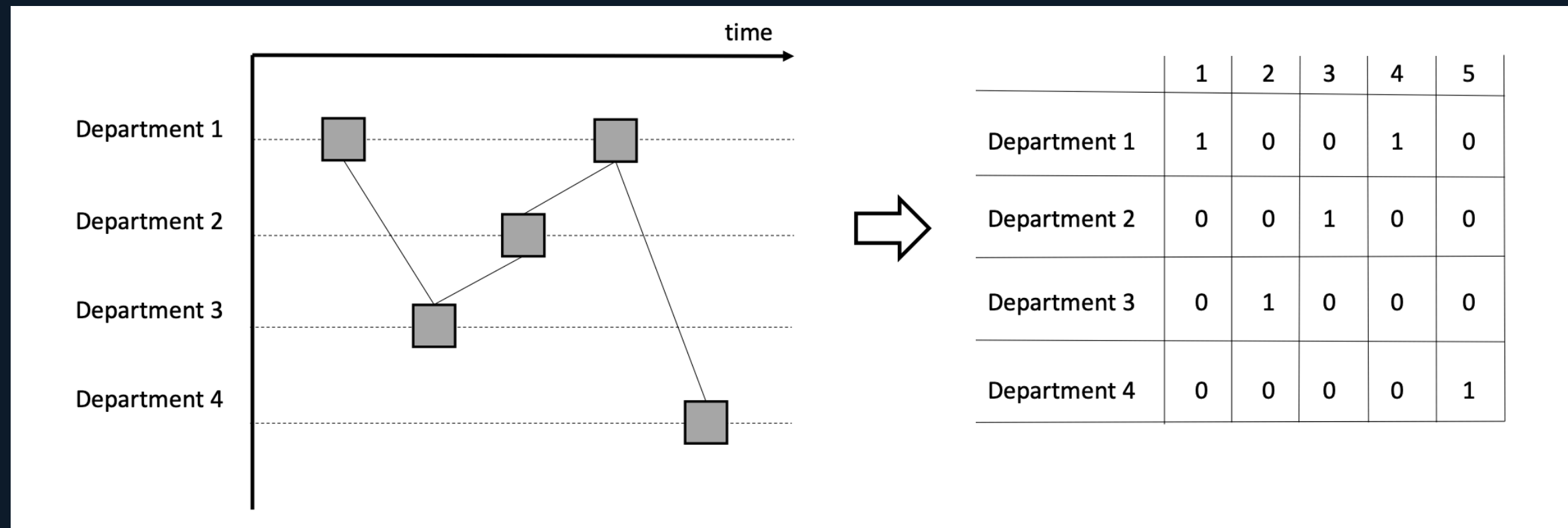
Tarefas:

1. Preencha as tabelas DRASiW (contadores) para RAM_0 e RAM_1
2. Calcule os valores B_{ij} para cada bit (b0, b1, b2, b3)
3. Ordene os pixels por frequência — qual seria o protótipo da Classe “X”?

Parte III — Bleaching

Solução para o Sobretreinamento

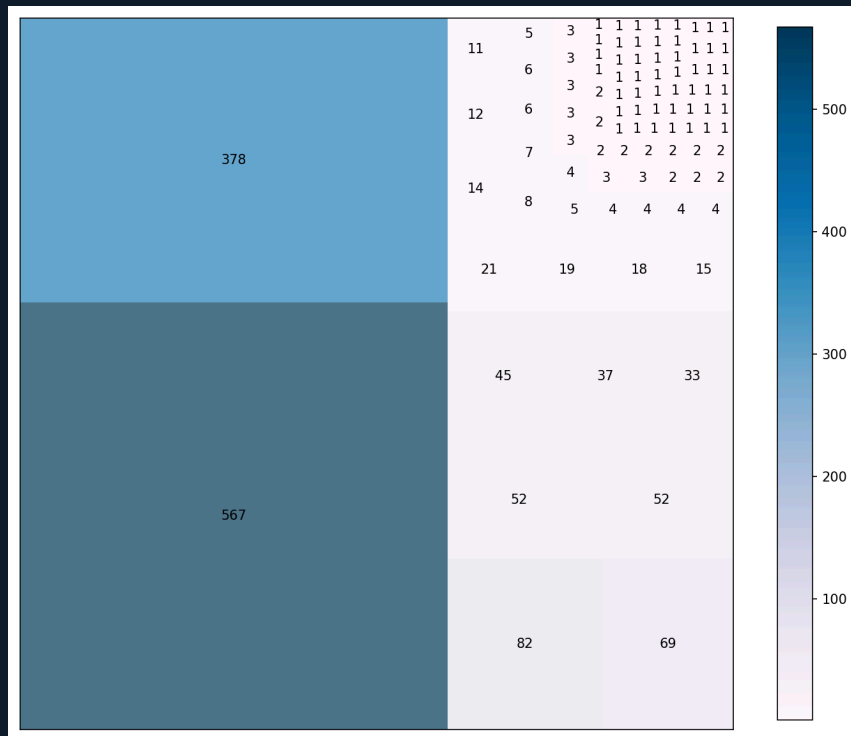
Overfitting ou Sobretreinamento - Um exemplo



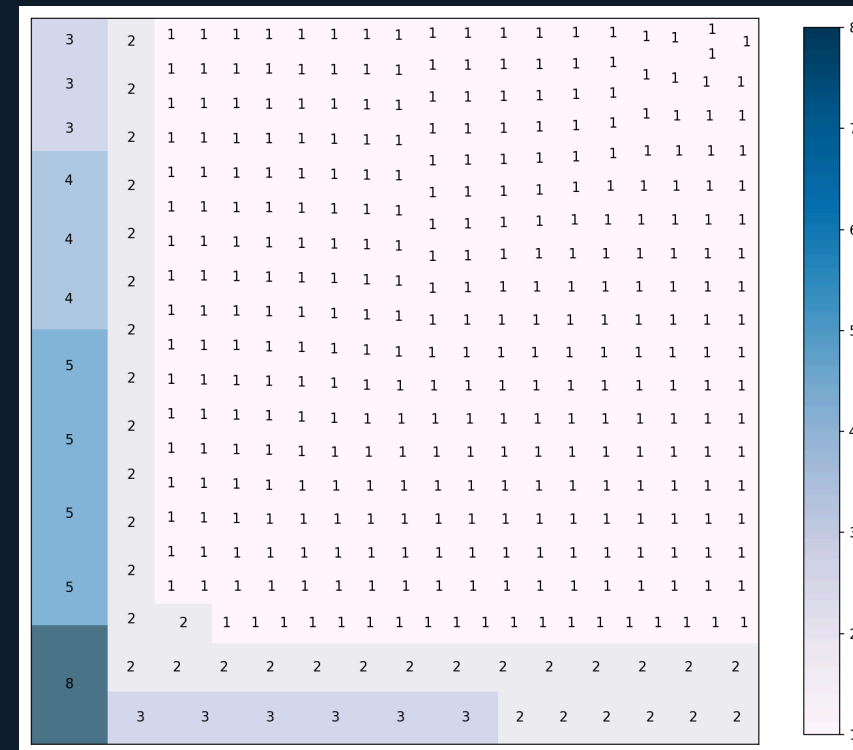
Retina de um Grafo através da matriz sequencial (similar a de adjacência) do grafo ($D \times S$), **Carvalho, D. et al.** (2020)

Entropia de Shannon das classes Carvalho, D. et al. (2020)

$$H(X) := - \sum_{x \in \mathcal{X}} p(x) \log p(x)$$



Classe A, $H = 2.877$



Classe B, $H = 8.584$

O Problema do Sobretreinamento

O WiSARD clássico sofre de **saturação** após muitos exemplos de treino:

Sintoma

RAMs preenchidas com 1s

$r_A \approx r_B \approx r_C \approx X$

Empates frequentes

Confiança $c \rightarrow 0$

Causa

Subpadrões raros e frequentes

têm o mesmo peso (bit = 1)

Perda de discriminação

entre classes similares

Solução proposta — Bleaching:

*Usar a informação de **frequência** (do DRASiW) para eliminar subpadrões pouco representativos, resolvendo empates de forma progressiva.*

Bleaching — Definição Formal

Parâmetro: limiar de bleaching $b \geq 1$

Operação: durante a classificação, uma posição RAM só contribui com 1 se seu contador $\geq b$:

$$\text{RAM}_i^{(b)}[\text{end}] = \begin{cases} 1 & \text{se contador}[\text{end}] \geq b \\ 0 & \text{caso contrário} \end{cases}$$

Resposta com bleaching:

$$r^{(b)} = \sum_{i=1}^X \text{RAM}_i^{(b)}(\text{n-tupla}_i)$$

Algoritmo sequencial (original):

```
b ← 1
enquanto houver empate nas respostas dos discriminadores:
    b ← b + 1
    recalcular  $r^{(b)}$  para todos os discriminadores
classificar com o discriminador de maior  $r^{(b)}$ 
```

Visualizando o Bleaching

Exemplo: $X=4, n=3, b=1$ (Carvalho et al., ESANN 2013)

RAM ₀			RAM ₁			RAM ₂			RAM ₃		
end.	cnt	b=1	end.	cnt	b=1	end.	cnt	b=1	end.	cnt	b=1
000	1	1	000	0	0	000	2	1	000	0	0
001	2	1	001	3	1	001	1	1	001	2	1
010	0	0	010	1	1	010	0	0	010	1	1
...			...			...			...		

Exemplo: $X=4, n=3, b=2$ (Carvalho et al., ESANN 2013)

RAM ₀			RAM ₁			RAM ₂			RAM ₃		
end.	cnt	b=2	end.	cnt	b=2	end.	cnt	b=2	end.	cnt	b=2
000	1	0	000	0	0	000	2	1	000	0	0
001	2	1	001	3	1	001	1	0	001	2	1
010	0	0	010	1	0	010	0	0	010	1	0
...			...			...			...		

$r_T^{(b=2)} = 4$ (RAM₀ + RAM₁ + RAM₂ + RAM₃ com contadores ≥ 2)

Três Estratégias de Bleaching

Carvalho et al. (ESANN 2013) comparam três abordagens:

1. Sequencial

$b = 1, 2, 3, \dots$

incrementa 1 por 1

até eliminar empates

-
- ✓ Simples
 - ✓ Sempre encontra solução
 - ✗ Lento em datasets grandes
 - ✗ Muitas iterações

2. Binário (B-bleaching)

Busca binária no espaço de b

$b \in [b_{\min}, b_{\max}]$

$O(\log n)$ iterações

-
- ✓ Muito mais rápido
 - ✓ Sem perda de acurácia
 - ✓ Ideal para online learning
 - ✗ Ligeiramente mais complexo

3. Por Confiança

Maximiza a confiança c

em vez de apenas resolver
empates

$$c = (r_T - r_I) / r_T$$

-
- ✓ Maximiza qualidade
 - ✓ Melhor separação
 - ✗ Mais iterações que binário
 - ✗ Nem sempre converge rápido

B-bleaching — Busca Binária

Motivação: o bleaching sequencial pode exigir muitas iterações quando b^* é grande.

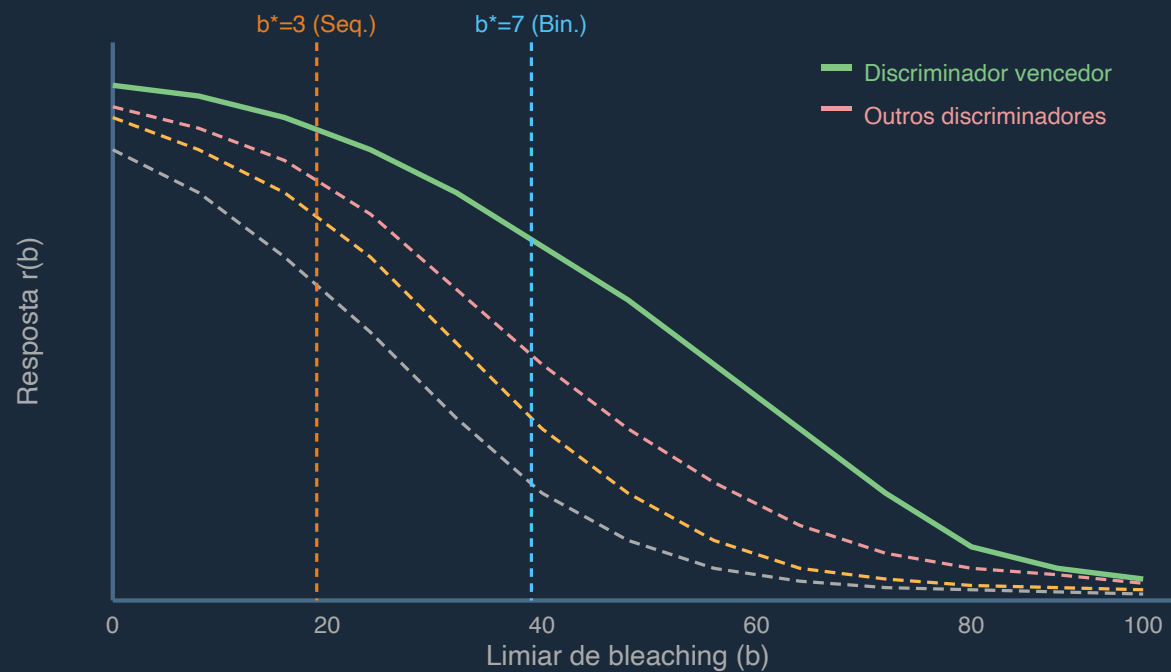
Algoritmo B-bleaching

```
// b_min = menor contador  $\neq 0$  encontrado nas RAMs
// b_max = maior contador encontrado nas RAMs
b_low  $\leftarrow$  b_min
b_high  $\leftarrow$  b_max
enquanto b_low < b_high:
    b_mid  $\leftarrow$  (b_low + b_high) // 2
    se empate_com(b_mid):
        b_low  $\leftarrow$  b_mid + 1    // aumenta limiar
    senão:
        b_high  $\leftarrow$  b_mid      // reduz limiar
classificar com b = b_low
```

Resultado (Carvalho et al., 2013): B-bleaching reduz o número de iterações **significativamente** (ordem de $\log_2(b^*)$ vs b^*) **sem perda de acurácia.**

Curvas de Resposta do Discriminador

Comportamento típico de $r^{(b)}$ conforme b aumenta:



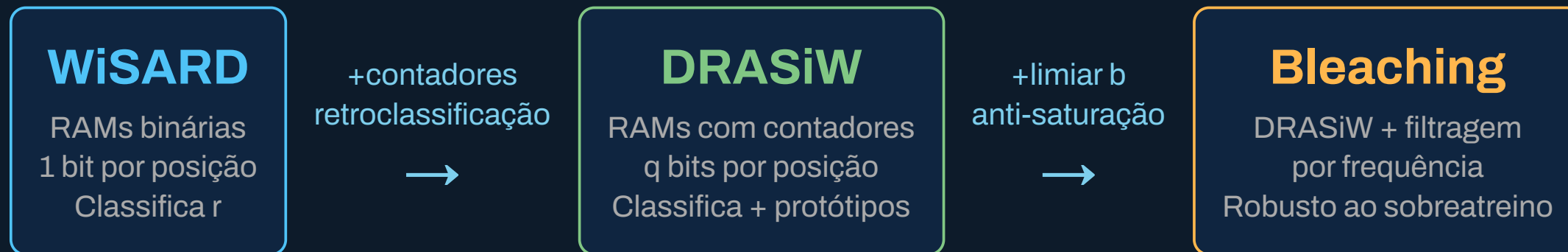
Adaptado de Carvalho et al. (ESANN 2013, Fig. 3)

Comparativo das Três Estratégias

Métrica	Sequencial	B-bleaching	Confiança
Iterações médias	b^*	$\lceil \log_2 b^* \rceil$	$b^* + 1$ extra
Acurácia	Base	Equivalente	Equivalente
Resolução de empates	Sempre	Sempre	Sempre
Maximiza confiança c	Não	Não	Sim
Complexidade	$O(b^*)$	$O(\log b^*)$	$O(b^*)$
Indicado para	Protótipos simples	Online learning	Alta confiança

*B-bleaching é a **estratégia recomendada** quando velocidade é crítica — como em aprendizado online e aplicações em tempo real (visão computacional, controle de robôs).*

Relação entre os Três Modelos



Linha evolutiva: WiSARD (1984) → DRASiW (1998) → Bleaching (2010s)

Cada extensão **adiciona capacidade** sem abandonar o paradigma sem-peso.

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

- Tempo real exigido (sem backpropagation)

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

- Tempo real exigido (sem backpropagation)
- Aprendizado online (novos exemplos incrementais)

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

- Tempo real exigido (sem backpropagation)
- Aprendizado online (novos exemplos incrementais)
- Hardware simples (RAM convencional)

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

- Tempo real exigido (sem backpropagation)
- Aprendizado online (novos exemplos incrementais)
- Hardware simples (RAM convencional)
- Interpretabilidade dos protótipos (DRASiW)

Aplicações Práticas

Domínios explorados nos artigos:

- **Reconhecimento de imagens:** MNIST, faces, dígitos manuscritos (Grieco et al., 2010)
- **Visão robótica:** rastreamento de pontos em plataformas offshore (França et al., ICARCV 2010)
- **Bioinformática:** categorização de subtipos do HIV-1 (Souza, COPPE 2011)
- **PLN:** categorização gramatical multilíngue (Carvalho et al., ESANN 2013)

Por que WiSARD/DRASiW nestas aplicações?

- Tempo real exigido (sem backpropagation)
- Aprendizado online (novos exemplos incrementais)
- Hardware simples (RAM convencional)
- Interpretabilidade dos protótipos (DRASiW)
- Robustez ao ruído (com bleaching)

Atividade 3 — Bleaching na Prática

Duração: 20 minutos | Em grupo (3–4 pessoas)

Cenário: WiSARD com DRASiW treinado, $X=3$ RAMs, $n=2$ bits

Contadores após treino (Discriminador A):

Endereço	RAM ₀	RAM ₁	RAM ₂
00	5	3	4
01	1	0	2
10	0	4	1
11	3	2	5

Padrão de teste endereça: RAM₀→ 11 , RAM₁→ 00 , RAM₂→ 11

Tarefas:

1. Calcule $r^{(b)}$ para $b = 1, 2, 3, 4, 5$
2. Outro discriminador B tem $r_B^{(b=1)} = 3$ e $r_B^{(b=3)} = 1$ — em que b o empate é resolvido?
3. Quantas iterações usa o bleaching **sequencial** e o **B-bleaching** para chegar ao mesmo resultado?
4. Discuta: quando o bleaching excessivo (b muito alto) pode prejudicar a generalização?

Discussão Final — Questões para Reflexão

Questões conceituais:

1. Por que o WiSARD não precisa de gradiente para aprender? Quais são as implicações práticas?
2. Como o tamanho n das n -tuplas afeta o tradeoff viés-variância no WiSARD?
3. Em que sentido os “protótipos” do DRASiW são análogos ao que um humano faz ao tentar lembrar a aparência de um número?
4. O bleaching é uma forma de **regularização**. Relacione com outros métodos de regularização que você conhece (L1, L2, dropout).

Questões de pesquisa:

5. Quais são as limitações da WiSARD comparada a redes convolucionais profundas para reconhecimento de imagens?
6. Como poderíamos combinar WiSARD com embeddings de entrada (e.g., word2vec) para PLN?
7. Seria possível implementar um WiSARD em hardware FPGA para processamento de vídeo em tempo real? Quais os desafios?
8. Como o bleaching se comporta em **aprendizado contínuo** (continual learning) com concept drift?

Resumo da Aula

WiSARD

- Neurônio RAM: n-tupla $\rightarrow$ endereço
- Discriminador: conjunto de RAMs + soma
- Multi-discriminador: 1 por classe
- Confiança $c = (r_T - r_I) / r_T$
- Aprendizado = escrita de 1s

DRASiW

- Contadores inteiros (q bits)
- Frequência dos subpadrões
- Retroclassificação: imagem mental
- Equivalência com regras fuzzy
- Base para o bleaching

Bleaching

- Limiar b: filtra subpadrões raros
- Resolve saturação/sobretreinamento
- Sequencial: $b = 1, 2, 3, \dots$
- B-bleaching: busca binária $O(\log b^*)$
- Por confiança: maximiza c

Referências Principais

- **Aleksander, I. & Morton, H.** (1990). *An Introduction to Neural Computing*. Chapman & Hall.
- **Soares, C.M. et al.** (1998). Uma Implementação em Software do Classificador WISARD. *V Simpósio Brasileiro de Redes Neurais*, Belo Horizonte.
- **Grieco, B.P.A., Lima, P.M.V., De Gregorio, M. & França, F.M.G.** (2010). Producing pattern examples from “mental” images. *Neurocomputing*, 73, 1057–1064.
- **França, H.L. et al.** (2010). Movement Pursuit Control of an Offshore Automated Platform via a RAM-based Neural Network. *ICARCV 2010*, Singapura.
- **Souza, C.R.** (2011). *Redes Neurais Sem-Peso Aplicadas na Categorização de Subtipos do HIV-1*. Dissertação de Mestrado, COPPE/UFRJ.
- **Carvalho, D.S. et al.** (2013). B-bleaching: Agile Overtraining Avoidance in the WiSARD Weightless Neural Classifier. *ESANN 2013*, Bruges.
- **Carvalho, D. et al.** (2020). Process Mining Classification with a Weightless Neural Network. *IJCIEOM 2020*, Rio de Janeiro.

Obrigado!

Dúvidas e discussões são bem-vindas

Professor
Diego Carvalho

Contato
diego.carvalho@cefet-rj.br